INFORME

Práctica 13 - Red neuronal aplicada

Alumnas: Mia Bertola, Julieta Barahona Roth y Valentina Calzetta

1. Problema y objetivo

En esta práctica entrenamos una red neuronal pequeña para aproximar una relación sintética asociada a la predicción de demanda. Retomamos el contexto de las prácticas anteriores, en las que analizamos solicitudes, colas y capacidad de atención. En la práctica 12 construimos una regresión lineal y la comparamos con una referencia constante. En esta oportunidad incorporamos una red neuronal para estudiar si una alternativa más compleja nos permite mejorar las predicciones.

La pregunta que nos planteamos fue: ¿podemos obtener menor error con la red neuronal que con una regresión lineal o con la repetición del último valor observado, y justificar así su complejidad?

Para responderla, completamos la arquitectura mínima del manual, separamos los datos de entrenamiento y prueba y evaluamos los tres métodos sobre las mismas muestras. También revisamos el efecto de aumentar el entrenamiento y desarrollamos un desafío en el que eliminamos la oscilación de los datos. Obtuvimos evidencia de que la red aprende, aunque no superamos al baseline en el caso base ni a la regresión en ninguno de los dos escenarios estudiados con el presupuesto principal.

2. Datos y supuestos

Generamos cien pares de observaciones mediante las funciones incluidas en el HTML del manual:

```
x[i] = i / 100, para i = 0, 1, …, 99
y = 0.2 + 0.6*x + 0.15*sin(8*x)
```

Combinamos así una tendencia creciente con una oscilación. A diferencia de la práctica 12, no incorporamos ruido: trabajamos con datos determinísticos y no necesitamos una semilla aleatoria. Al repetir el procedimiento con la misma configuración, obtenemos las mismas observaciones.

Conservamos la escala pequeña que propone el manual para el entrenamiento. Aunque allí encontramos la descripción de un conjunto normalizado, no contamos con una transformación que vincule estos valores con cantidades reales de solicitudes. Por eso trabajamos con unidades abstractas y no interpretamos los errores como solicitudes de una plataforma real. Tampoco ajustamos otro escalador sobre el conjunto completo.

Organizamos los datos de la siguiente manera:

Conjunto	Índices	Valores de x	Cantidad	Uso
Entrenamiento	0 a 79	0,00 a 0,79	80	Ajustar los modelos y obtener el último valor observado
Prueba	80 a 99	0,80 a 0,99	20	Evaluar las predicciones

Conservamos el orden de las muestras, siguiendo el criterio de separación utilizado en la práctica 12. Ajustamos los modelos solamente con entrenamiento y no incorporamos las respuestas de prueba a las actualizaciones. Como todos los valores de x de prueba superan a los de entrenamiento, evaluamos extrapolación fuera del intervalo observado.

Usamos datos distintos de los de la práctica 12 y en otra escala. Por ese motivo, comparamos nuevamente la regresión sobre los datos de esta práctica; no contrastamos directamente sus MAE con los del informe anterior.

3. Métodos y medición del error

Construimos el baseline repitiendo el último valor de entrenamiento durante las veinte muestras de prueba. Con esta referencia representamos el supuesto de que el valor futuro se mantendrá en el último nivel observado.

Para la regresión lineal ajustamos por mínimos cuadrados la relación:

```
y estimada = intercepto + pendiente*x
```

Calculamos ambos coeficientes con las ochenta muestras de entrenamiento. Después extendimos la recta a las veinte entradas reservadas, sin actualizar sus parámetros.

Para la red neuronal conservamos una entrada, una neurona oculta con activación sigmoide y una salida sin sesgo adicional, tal como propone el código inicial:

```
h = sigmoid(w1*x + b)
predicción = w2*h
```

Inicializamos los parámetros con `w1 = 0.1`, `w2 = −0.2` y `b = 0.0`. Usamos una tasa de aprendizaje de `0.1` y fijamos un presupuesto principal de 10.000 épocas. En cada época recorrimos las ochenta muestras, promediamos los gradientes y actualizamos simultáneamente los tres parámetros. Implementamos así descenso por gradiente de lote completo.

Minimizamos la mitad del error cuadrático medio. Para una muestra, calculamos:

```
error = predicción − y
```

```
dL/dw2 = error*h
dL/dw1 = error*w2*h*(1-h)*x
dL/db  = error*w2*h*(1-h)
```

Promediamos esas contribuciones sobre entrenamiento y aplicamos la actualización `parámetro nuevo = parámetro actual - tasa de aprendizaje*gradiente`. Evaluamos todas las derivadas con los mismos parámetros antes de actualizarlos.

Medimos el desempeño final de los tres métodos con el error absoluto medio:

```
MAE = suma de |valor real - valor predicho| / cantidad de
```

Interpretamos un MAE menor como un menor desvío absoluto promedio en la escala de los datos. También registramos el error cuadrático medio, o MSE, para analizar el aprendizaje. Distinguimos así el objetivo usado para entrenar de la medida principal con la que comparamos los modelos.

4. Caso base y comparación con la regresión lineal

Con el baseline mantuvimos el último valor de entrenamiento, aproximadamente `0,679521`. Con la regresión obtuvimos la siguiente recta, mostrando los coeficientes redondeados:

```
y estimada = 0.336183 + 0.255359*x
```

Después de 10.000 épocas obtuvimos la siguiente red:

```
y estimada = 0.806018 * sigmoid(1.079474*x - 0.249493)
```

Usamos los parámetros sin redondear para calcular las predicciones y las métricas.

Método	MAE sobre las veinte muestras de prueba	MSE sobre las veinte muestras de prueba
Baseline constante	0,160901	0,031791
Regresión lineal	0,275692	0,079884
Red neuronal	0,299004	0,093742

Obtuvimos el menor error con el baseline. Con la red registramos un MAE aproximadamente un 85,83 % mayor que con el baseline y un 8,46 % mayor que con la regresión lineal. Calculamos estos porcentajes a partir de los valores sin redondear.

Al comparar los ajustes sobre entrenamiento, obtuvimos un MAE de `0,060147` con la red y de `0,060601` con la regresión. Aunque conseguimos un MAE ligeramente menor con la red en ese tramo, no mantuvimos esa ventaja en prueba. Con esto comprobamos que ajustar mejor los ejemplos observados según una métrica no garantiza mejores predicciones fuera de ellos.

[image]

Al revisar las predicciones individuales, encontramos que para `x = 0,40` estimamos `0,439642` con la red frente a un valor real de `0,431244`. En cambio, para `x = 0,99` estimamos `0,559428` frente a `0,943673`. En ese último punto obtuvimos `0,588989` con la regresión y `0,679521` con el baseline. Observamos así una subestimación marcada del ascenso final.

Relacionamos este comportamiento con la estructura del modelo. Con una única sigmoide y una salida de la forma `w2*h`, solo podemos producir una función monótona o constante: el signo de su derivada depende de `w1*w2` y no cambia con x. Por eso no podemos reproducir todos los ascensos y descensos de la relación del manual con esta arquitectura.

También reconocemos que el baseline queda mejor ubicado para este horizonte concreto al conservar el último nivel observado. No interpretamos su ventaja como una garantía de superioridad frente a

otros cortes o patrones de datos.

5. Curvas de aprendizaje y comparación del presupuesto de entrenamiento

Registramos los parámetros y el MSE cada cien épocas, incluyendo la época inicial y la final. Con el presupuesto principal redujimos el MSE de entrenamiento de `0,298874` a `0,004832`, aproximadamente un 98,38 %. Con esta disminución comprobamos que el entrenamiento modifica los parámetros de forma útil para reducir su pérdida.

[image]

Incluimos también el MSE de prueba como diagnóstico de los parámetros guardados. No lo usamos para actualizar pesos ni para seleccionar automáticamente la época final. Al interpretar la curva, distinguimos la disminución del error sobre los ejemplos conocidos del desempeño sobre entradas que quedan fuera de su rango.

Para revisar si el presupuesto elegido condicionaba la conclusión, realizamos después una corrida adicional de 100.000 épocas, manteniendo los datos, los parámetros iniciales y la tasa de aprendizaje.

Presupuesto de la red	MSE de entrenamiento	MAE de prueba
10.000 épocas	0,00483230	0,299004
100.000 épocas	0,00474506	0,278376

Obtuvimos una mejora al extender el entrenamiento. Por lo tanto, no afirmamos que 10.000 épocas demuestren convergencia ni que constituyan el mejor presupuesto posible. Aun con 100.000 épocas, mantuvimos un MAE de prueba superior al del baseline, de `0,160901`, y ligeramente superior al de la regresión, de `0,275692`.

Consideramos esta extensión un diagnóstico posterior. Como ya habíamos inspeccionado el conjunto de prueba, no la presentamos como una selección de hiperparámetros validada. Para elegir una configuración y medir su generalización necesitaríamos separar un conjunto de validación y reservar otra evaluación independiente.

6. Desafío adicional: relación sin oscilación

Decidimos modificar la estructura de los datos para estudiar cuánto dependía el resultado de la relación que intentábamos aprender. Eliminamos el componente periódico y conservamos únicamente:

```
y = 0.2 + 0.6*x
```

Mantuvimos las mismas entradas, el corte de ochenta y veinte muestras, la arquitectura, los parámetros iniciales, la tasa de aprendizaje y las 10.000 épocas del caso principal. Volvimos a ajustar cada modelo desde el inicio con los nuevos datos de entrenamiento.

Con este desafío cambiamos la relación en todo el conjunto. No introdujimos un salto sorpresivo al comenzar la prueba, como hicimos en la práctica 12. Buscamos aislar el efecto de eliminar la oscilación sobre el aprendizaje y la comparación entre modelos.

Método	MAE de prueba con oscilación	MAE de prueba sin oscilación
Baseline constante	0,160901	0,063000
Regresión lineal	0,275692	0,000000
Red neuronal	0,299004	0,015417

Conseguimos mejorar considerablemente el resultado de la red y superar al baseline en el escenario sin oscilación. Sin embargo, con la regresión recuperamos exactamente el intercepto `0,2` y la pendiente `0,6`, y obtuvimos error cero al continuar una relación lineal sin ruido.

Interpretamos que el método más conveniente depende de la estructura de los datos. En el desafío encontramos una ventaja clara de la regresión: representamos exactamente la relación con un ajuste sencillo, mientras que con la red obtuvimos una aproximación y necesitamos entrenamiento iterativo.

7. Verificación y evidencia

Comprobamos la predicción inicial con cálculos manuales. Para $x = 0$, por ejemplo, obtuvimos $h = 0{,}5$ y una salida de $-0{,}1$ con los parámetros iniciales. También contrastamos el cálculo del MAE y del MSE con un ejemplo pequeño de respuesta conocida.

Revisamos los gradientes analíticos mediante diferencias finitas en distintas configuraciones de pesos. Además, implementamos en las pruebas un entrenamiento independiente que aproxima numéricamente las derivadas, sin usar las funciones de predicción ni de gradiente del programa principal. Después de 2.000 épocas obtuvimos una diferencia máxima entre los pesos de aproximadamente $3{,}74 \times 10^{-11}$.

Para verificar la separación de datos, alteramos las respuestas de prueba y ejecutamos la evaluación completa. Comprobamos que cambiaran los errores de prueba, pero no los pesos de la red, los coeficientes de la regresión ni el valor del baseline. Con esto verificamos que no usamos las respuestas reservadas para ajustar los modelos.

Ejecutamos cinco pruebas automáticas y obtuvimos resultados satisfactorios en todas. También comprobamos que cada escenario tuviera cien registros, veinte de ellos reservados, que las predicciones fueran finitas y que al repetir la ejecución generáramos archivos idénticos.

Guardamos la evidencia en la carpeta `resultados/`:

- En `base_predicciones.csv` registramos los valores reales, la partición, las tres predicciones y los errores absolutos de la red.
- En `base_aprendizaje.csv` conservamos las épocas, los pesos y los MSE de entrenamiento y prueba.
- En `base_predicciones.svg` y `base_aprendizaje.svg` representamos las predicciones y las curvas de aprendizaje.
- En los archivos con prefijo `sin_oscilacion` guardamos la misma evidencia para el desafío.
- En `resumen.json` registramos la configuración principal, los parámetros, las métricas de los dos escenarios y las comprobaciones internas.

Documentamos por separado el diagnóstico de 100.000 épocas en `ANALISIS.md`; no reemplazamos con él los resultados principales de 10.000 épocas guardados en `resumen.json`.

Para regenerar los resultados principales y ejecutar las pruebas usamos los siguientes comandos desde la carpeta general del repositorio:

```
python3 "Práctica 13/practica_13_red_neuronal.py"
python3 -m unittest discover -s "Práctica 13" -p "test_*.
```

No necesitamos instalar librerías externas. Conservamos las constantes de configuración al comienzo del programa para facilitar la repetición de los experimentos.

8. Limitaciones y conclusiones

Trabajamos con una relación sintética, determinística y de una sola variable. Evaluamos un único corte de entrenamiento y prueba, sin ruido, mediciones reales ni variables de calendario. Por eso limitamos nuestras conclusiones a estos escenarios y a las configuraciones estudiadas.

Reconocemos dos limitaciones distintas: con una sola sigmoide no podemos representar toda la oscilación y, al reservar las últimas veinte entradas, debemos extrapolar fuera del rango observado. También comprobamos que el presupuesto de entrenamiento influye en el resultado. Por estas razones, no atribuimos toda la diferencia de error a una única causa ni diagnosticamos sobreajuste solamente a partir de la diferencia entre entrenamiento y prueba.

No comparamos redes con varias neuronas, otras inicializaciones ni una búsqueda sistemática de tasas de aprendizaje. Tampoco establecimos un umbral operativo de aceptación o evaluamos cómo se traducen los errores en costos, colas o incumplimientos de servicio.

Nuestra expectativa de mejorar la predicción por incorporar una red neuronal no se cumplió en el caso base. Aunque redujimos la pérdida de entrenamiento y verificamos la implementación, obtuvimos menor MAE de prueba con el baseline. Al eliminar la oscilación mejoramos la

red y superamos esa referencia, pero con la regresión lineal obtuvimos el mejor resultado posible para esa relación exacta.

Concluimos que no encontramos una justificación para elegir esta red sobre los métodos simples en los escenarios evaluados. No extendemos esta conclusión a cualquier arquitectura neuronal: comprobamos el desempeño de la red mínima del manual y de los presupuestos que analizamos.

Podríamos usar estos resultados para seleccionar un predictor antes de integrarlo con el simulador de la práctica 14. Sin embargo, todavía no determinamos cuántos servidores necesitamos. Para apoyar esa decisión tendríamos que relacionar la demanda predicha con la capacidad de atención, definir un margen de error aceptable y evaluar las consecuencias de subestimar la carga. Antes de aplicarlo a una plataforma real, también necesitaríamos validar el enfoque con registros observados.